Use of AI assistants. Problems 8.1 to 8.4 and the bonus are No AI. Problem 8.5 marks **each of its parts separately**, under the rules printed with it. Answers go in **one Haskell file**, built from the template: code as code, types, predictions and explanations in comments. At a defense you may be asked to write or trace any function from this sheet on paper.

Two rules apply to every problem. **No partial functions**: a function without an equation for some shape of its input, or a call to `head`, `tail` or `!!`, loses the working points of its part. **The file loads**: a part you have not finished stays `undefined`, or in a comment.

Consider a service of short links, such as `bit.ly`. A request `PUT /ghcup https://...` makes the short link `/ghcup` to an `https` address, if no link has that alias yet, and `GET /ghcup` redirects to the address and counts a click. A link works for 30 days: made on 1 Oct, it redirects up to 30 Oct and is gone from 31 Oct. The template gives these types and functions:

```haskell
data Method = GET | PUT
  deriving (Show, Read, Eq)
data LinkRequest
  = Create String String  -- PUT /alias url
  | Follow String         -- GET /alias
  deriving (Show, Eq)
data Link = Link
  { alias :: String, target :: String
  , created :: Day, clicks :: Int }
  deriving (Show, Read, Eq)
```

```haskell
data Response
  = Created               -- 201
  | Redirect String       -- 302, to this url
  | BadRequest String     -- 400, with a reason
  | NotFound              -- 404
  | Conflict              -- 409, the alias is taken
  | Gone                  -- 410, the link expired
  deriving (Show, Eq)
alive :: Day -> Link -> Bool
findLink :: String -> [Link] -> Maybe Link
countClick :: String -> [Link] -> [Link]
statusLine :: Response -> String
```

`alive d l` says whether `l` works on the day `d`. From `Data.Time`, `getCurrentTime :: IO UTCTime` reads the clock, and `utctDay :: UTCTime -> Day` gives the date. `readMaybe` reads a `Day` written as `2026-10-14`, and `oct 14` is 14 Oct 2026.

Problem 8.1. No AI Consider the first version, which reads the clock where it needs the date:

```haskell
follow :: String -> [Link] -> IO [Link]
follow a ls = do
  now <- getCurrentTime
  let today    = utctDay now
      notFound = putStrLn "404 Not Found"
  case findLink a ls of
    Nothing -> do
      notFound
      pure ls
    Just l ->
      if alive today l
        then do
          putStrLn ("302 Found: " ++ target l)
          pure (countClick a ls)
        else do
          putStrLn "410 Gone"
          pure ls
```

(a) Give the type of each of these in `follow`, and say whether it is an action or a plain value: `now`, `today`, `notFound`, `findLink a ls`, `alive today l`, `pure ls`, `follow "ghcup"`, and the whole `else` block.

(b) Write `follow` with `>>=`, `>>` and lambdas, and no **do**.

(c) The template's `someLinks` holds `"ghcup"`, made on 14 Oct with no clicks, and `"lyah"`, made on 2 Sep with five. The template also gives this program:

```haskell
followMain :: IO ()
followMain = do
  ls1 <- follow "ghcup" someLinks
  ls2 <- follow "ghcpu" ls1
  ls3 <- follow "lyah" ls2
  ls4 <- follow "ghcup" ls3
  print (map clicks ls4)
```

What does `followMain` print on 19 Oct, and on 14 Nov? How many times do `getCurrentTime` and `notFound` run? Why does the template give no expected output for `followMain`?

(d) Does GHC accept each change to `follow`? If it does, what does `followMain` print on 19 Oct?

1. `now <- getCurrentTime` becomes `now = getCurrentTime` in the **let**.
2. `pure ls` in the **Nothing** branch becomes `ls`.
3. `notFound` moves from the **Nothing** branch to just before the **case**.

Problem 8.2. No AI Consider the service's parser, given as a staircase. The given `aliasOf` and `secure`, of type **String -> Maybe String**, give the alias in a path such as `/ghcup` and an address that starts with `https://`.

```haskell
parseLinkRequest :: String -> Maybe LinkRequest
parseLinkRequest line =
  case words line of
    (m : p : more) ->
      case readMaybe m of
        Nothing -> Nothing
        Just meth ->
          case aliasOf p of
            Nothing -> Nothing
            Just a ->
              case (meth, more) of
                (GET, [])    -> Just (Follow a)
                (PUT, [url]) ->
                  case secure url of
                    Nothing -> Nothing
                    Just u  -> Just (Create a u)
                _ -> Nothing
    _ -> Nothing
```

(a) Rewrite `parseLinkRequest` with **do** in **Maybe**, and with no **case** on a **Maybe**.

(b) Write `readLinkRequest`, which gives a **Right** where `parseLinkRequest` gives a **Just**, and a **Left** with the reason where it gives **Nothing**:

```haskell
readLinkRequest "POST /ghcup"   -- Left "no such method: POST"
```

(c) The service keeps a log, a line per request: its date, a space and the request, such as `2026-10-14 GET /ghcup`. Write `readLog`, of the type below, by explicit recursion. It gives every entry, or the first line whose date cannot be read, with its number, and it does not check the requests. Then explain in two sentences why a bad date refuses the whole log, while a bad request in a line does not.

```
readLog :: [String] -> Either String [(Day, String)]
readLog ["2026-10-14 GET /ghcup", "2026-11-31 GET /ghcup"]
-- Left "line 2: no date: 2026-11-31"
```

Problem 8.3. No AI Consider the rules of the service once more, with no clock and no printing.

(a) Write the two functions below. `handle` does what `follow` did, and handles **`Create`** too, with the day of the request as the new link's date. `respond` reads the line with `readLinkRequest`, and answers a line it cannot read with a **`BadRequest`** and the links as they were.

```
handle  :: Day -> LinkRequest -> [Link] -> (Response, [Link])
respond :: Day -> String -> [Link] -> (Response, [Link])
```

(b) Write, by explicit recursion, the function of this type, which runs a step on each input in turn, passes the state on, and collects the outputs:

```
replay :: (r -> s -> (a, s)) -> [r] -> s -> ([a], s)
replay (\x n -> (x * n, n + 1)) [10, 10, 10] 1   -- ([10,20,30],4)
```

Then say what `replay entry` does, where `entry (d, line) ls = respond d line ls`, and explain from its type why `replay` can neither read input nor print.

(c) A check is a name, and whether it holds: a plain value of type `(String, Bool)`. The template's `ghcupLink` was made on 1 Oct, and its first check is

```
( "an unknown alias is not found"
, handle (oct 5) (Follow "ghcpu") [ghcupLink] == (NotFound, [ghcupLink]) )
```

Add at least six checks of `handle`, `respond` and `replay`, which cover every response of `handle`, the last day a link works and the first day it is gone, and a session through `replay`. Then say in two sentences which checks could not be written for `follow`, and why.

Problem 8.4. No AI Consider the same core behind two shells.

(a) Write the function of this type. It runs the action for the next input, and stops with the state at **`Nothing`**. On **`Just`** `r`, it applies the step, prints its line, and goes on with the new state. Then say what the type lets `serveFrom` know about requests and links.

```
serveFrom :: IO (Maybe r) -> (r -> s -> (String, s)) -> s -> IO s
```

(b) Write `fromKeyboard :: IO (Maybe (Day, String))`, which reads a line and gives **`Nothing`** for an empty one, and otherwise the line with today's date. Then write `console :: IO ()`, which serves requests from the keyboard, starting with no links, and at the end prints each link with the template's `describeLink`.

(c) Write `replayFile :: FilePath -> FilePath -> IO ()`. It reads a log from the first file, and refuses it with the error of `readLog`. Otherwise it writes the status line of every response to the second file, and prints how many requests it replayed and the links.

(d) Write the sample files with the `files` mode, and run `console < requests-1.txt` and `replay log-1.txt out.txt` on the same requests. Answer with reasons.

1. On which lines do the console's responses and `out.txt` differ?
2. Delete the empty last line of `requests-1.txt`. What does the console print now?
3. Change the date on line 14 of the log to `2026-11-31`. What does the replay do, and what would the service have answered on that line?

Problem 8.5. Consider an assistant's version of the whole service.

(a) AI allowed Give an assistant this request:

> Write a Haskell console program for a link shortener. It reads one request per line from standard input. `PUT /alias https://...` makes a short link, and the alias must be new. `GET /alias` redirects to the address and counts a click. A link expires 30 days after it is made. Print one response line per request, with its HTTP status code. Stop at an empty line.

Use a **basic, free** model, and submit its program, verbatim, as `assistant.hs`.

(b) No AI Before running it, predict from its code what it prints on `requests-2.txt`: for each line, whether it reads the line and what it prints. Say where and how often it reads the clock, and mark every action that this input runs more than once, or never.

(c) No AI Run it on `requests-2.txt`, and your console too. Mark each line where your prediction and the run differ, and where the two programs differ, and say whether the request above decided it. Say which of your checks from 8.3(c) run on its program, and what function, with its type, its code would need for all of them to run. Then mark what is not idiomatic Haskell, as lectures 6 to 8 present it, with a line of reason each.

Use of AI assistants. Part (a) only. If you use an assistant, submit `transcript.md`: the whole exchange, verbatim and unedited. Do **not** assume that its program is correct; (b) and (c) are in your own words, and a wrong prediction in (b) reasoned from the code loses nothing. Without an assistant, write the program yourself.

Bonus. No AI **+0.5 points, extra.** Serve the links over HTTP with Scotty, or another web library from Hackage. In `server.hs`, a copy of your file, write `serve :: FilePath -> IO ()`. It answers `PUT /ghcup`, with the address as the request body, and `GET /ghcup` with `respond` and today's date. Store the links and their click counts in the file so they survive server restarts. A redirect sets the `Location` header. The template says how to run `server.hs` with `cabal`.